



PYTHON – TURTLE ART

GEOMETRIC PATTERNS

Session	Duration	Topic	Level	Tool
01	30 Minutes	Lines & Squares	Beginner	Trinket.io

■ Today's Digital Rangoli!

Today **YOU** will make the computer draw geometric art — like a **digital rangoli** — using Python! You will control a tiny robot tortoise called Turtle that holds a pen and draws beautiful shapes on the screen. Let's have fun creating art with code!

Inspired by the beautiful rangoli and kolam patterns drawn across India — from Diwali decorations to kolam on Tamil Nadu doorsteps!

■ What We Cover Today

1. What is Python?	pg 2
2. What is Turtle?	pg 2
3. Core Commands to Know	pg 3
4. Guided Demo — Drawing a Line	pg 3
5. Guided Demo — Drawing a Square	pg 4
6. Physical Activity — Walk the Square!	pg 5
7. Math Connection — The Full Circle	pg 5
8. Challenge Activities	pg 6
9. Review Quiz	pg 7
10. Wrap-Up, Homework & Indian Inspiration	pg 8



■ 1. What is Python?

Python is a programming language — a set of special words that computers understand. When you write Python code, you are giving the computer a step-by-step recipe to follow!

■ Indian Analogy ■

Think of Python like Mummy's recipe card — it tells exactly how to cook your favourite dish, step by step. The computer follows every single instruction without skipping anything, just like a good cook follows the recipe!

Key Facts

- Python uses English-like words
- It is one of the friendliest coding languages
- Used by scientists, artists and game makers
- Named after a comedy show, not the snake!

■ 2. What is Turtle?

Turtle is a special toolbox (called a **module**) inside Python that lets us draw pictures on the screen. It works by moving a small arrow — our "turtle" — around a blank canvas. Wherever the turtle walks, it draws a line!

■ The Pen

The turtle holds a pen. When the pen is down, it draws. When lifted up, it moves without drawing.

■ Start Position

The turtle always starts in the centre of the screen, facing right. All movements start from here.

■ Like Kolam!

Just like drawing kolam or rangoli patterns step by step, we tell the turtle to move step by step.

■ Think About It!

Have you seen someone draw a rangoli or kolam pattern? They move their hand step by step — forward a little, turn, forward again. That is **exactly** what we will tell the turtle to do using code! Can you think of any square shapes around you right now?



■ 3. Core Commands to Know

These are the six core commands you will use today. Read each one carefully — they are your turtle's instruction set!

`import turtle`

Wake up the turtle toolbox

`turtle.forward(100)`

Walk 100 steps forward

`turtle.right(90)`

Turn right by 90 degrees

`turtle.left(90)`

Turn left by 90 degrees

`turtle.penup()` / `pendown()`

Lift or place the pen

`turtle.done()`

Freeze the screen when finished

■ 4. Guided Demo — Drawing a Line

Let's open **Trinket.io** in your browser and type your first code together! Follow these three steps carefully, then press the green Run button.

1 Type: `import turtle`

2 Type: `turtle.forward(100)`

3 Type: `turtle.done()`

```
import turtle

turtle.forward(100)    # walk 100 steps
turtle.done()          # freeze screen
```

■ What Happened?

The number **100** inside `forward()` is called a **parameter**. It tells the turtle *how many* steps to walk. Change it to 200 and the line gets longer. Change it to 50 and it gets shorter. **You are in control!**

■ **Challenge:** Change `forward(100)` to `forward(200)` and run it. What do you notice?



■ 5. Guided Demo — Drawing a Square

A square has **4 equal sides** and **4 right angles (90 degrees)**. We will tell the turtle to walk forward and then turn right — four times! This is exactly like walking around the corner in Ludo!

Step 1 & 2 — First two sides:

1 `turtle.forward(100)` → Draw Side 1

2 `turtle.right(90)` → Turn Corner 1

3 `turtle.forward(100)` → Draw Side 2

4 `turtle.right(90)` → Turn Corner 2

Complete Square Code — all 4 sides:

```
import turtle

# Side 1
turtle.forward(100)
turtle.right(90)
# Side 2
turtle.forward(100)
turtle.right(90)
# Side 3
turtle.forward(100)
turtle.right(90)
# Side 4
turtle.forward(100)
turtle.right(90)

turtle.done()      # finished!
```



■ Brilliant! You're Coding a Shape!

Four sides. Four turns. A perfect square! Just like the floor tiles in your classroom or the squares on a carom board — your turtle has drawn one using only a few lines of Python code.

Try it yourself: After typing the full code, change `forward(100)` to `forward(150)`. What happens to the size of your square?

■ Bug Detective — Can you spot the mistake?

This code has ONE mistake. The square does not close properly. Can you find the bug?

```
import turtle
turtle.forward(100)
turtle.right(90)
turtle.forward(100)
turtle.right(45)
turtle.forward(100)
turtle.right(90)
turtle.forward(100)
turtle.done()
```

■ Hint: Look at line 5. It says `right(45)` — but a square needs `right(90)`. That tiny change breaks the whole shape! This is called a **bug**. Finding and fixing bugs is called **debugging**.



■ 6. Physical Activity — Walk the Square!

Stand up! We are going to **BE the turtle** and walk a square with our bodies. This is exactly what your turtle does in code!

Step 1	→	Face the front of the classroom.
Step 2	■	Walk 3 steps FORWARD.
Step 3	■	Turn RIGHT 90 degrees (like an autorickshaw at a junction).
Step 4	■	Walk 3 steps FORWARD again.
Step 5	■	Turn RIGHT 90 degrees.
Step 6	■	Walk 3 steps FORWARD.
Step 7	■	Turn RIGHT 90 degrees.
Step 8	■	Walk 3 steps FORWARD — you are back where you started!

■ 7. Math Connection — The Full Circle

Here is something amazing! Count up all the right turns in a square:

4 sides	×	90°	=	360°	= Full Circle!
---------	---	-----	---	------	----------------

Just like when an **autorickshaw** makes four turns at road junctions to go around a city block — or the corners of the tiles on your classroom floor — four right turns of 90° always bring you back to where you started. The turtle always turns a total of **360 degrees** to complete a closed shape!

■ Why Does This Work?

The total turning for any closed polygon is always 360°. For a square we split that into 4 equal turns of 90°. For a triangle it would be 3 turns of 120°. For a hexagon it would be 6 turns of 60°. **You will discover this yourself in the bonus challenge!**



■ 8. Challenge Activities

Activity 1 — Change Square Size!

What happens when you change **forward(100)** to **forward(150)**? Try it! Then try **forward(50)**. What do you notice?

The number inside **forward()** is called a **parameter**. Changing parameters changes the output. There are no wrong answers in art!

Try these values:

- **forward(50)** → tiny square
- **forward(100)** → medium square
- **forward(150)** → large square
- **forward(200)** → giant square!

Activity 2 — Early Finishers: Two Squares!

Can you make **TWO squares** sitting side by side? Use **penup()** to lift the pen, move without drawing, then **pendown()** to start drawing again. Here is the scaffold code:

```
import turtle

# --- Draw first square ---
turtle.forward(100)
turtle.right(90)
# ... (repeat 4 times total)

# --- Move to new position ---
turtle.penup()      # lift pen
turtle.forward(130) # move without drawing
turtle.pendown()    # place pen

# --- Draw second square ---
turtle.forward(100)
# ... (repeat 4 times total)
turtle.done()
```

Bonus Challenge — Draw a Hexagon! ■

Try using **turtle.right(60)** instead of **right(90)**. Repeat the forward + right pair **6 times**. What shape does the turtle draw? Hint: $6 \times 60^\circ = 360^\circ$ — a complete hexagon!



```
import turtle

# Hexagon: 6 sides, 60 degrees each
turtle.forward(80)
turtle.right(60)
turtle.forward(80)
turtle.right(60)
# ... repeat 6 times total
turtle.done()
```

■ Discovery!

You just discovered the secret formula for any regular polygon: **Number of sides × Turn angle = 360°**. Triangle: $3 \times 120^\circ$ | Square: $4 \times 90^\circ$ | Hexagon: $6 \times 60^\circ$. You are thinking like a real programmer!



■ 9. Review Quiz

Think carefully and answer these fun questions. Cover the answers and try first!

Q1 Which command makes the turtle move forward?

Answer: `turtle.forward()`

Q2 How many right turns does the turtle need to draw a square?

Answer: 4 times

Q3 What does `turtle.right(90)` mean?

Answer: Turn right by 90 degrees

Q4 What is the total degrees turned when drawing a square?

Answer: $4 \times 90 = 360$ degrees

Q5 What command lifts the pen so the turtle moves without drawing?

Answer: `turtle.penup()`

Q6 If you change `forward(100)` to `forward(50)`, what happens?

Answer: The square becomes smaller (half the size)

Q7 What is a "bug" in code?

Answer: A mistake in the code that makes it behave wrongly



■ Scoring

7/7 — You are a Python Master! ■■

5–6 — Brilliant work! Keep practising!

3–4 — Great effort — go back and try the demos again!

1–2 — No worries — every programmer starts from scratch!



■ 10. Wrap-Up — What We Learned Today

■ Python	A friendly language that computers understand — like a recipe for the computer.
■ Turtle	A robot artist that holds a pen and draws wherever it walks.
■ ■ 4 Commands	forward(), right(), left(), penup()/pendown(), done().
■ Drawing a Line	import turtle → turtle.forward(100) → turtle.done()
■ Drawing a Square	$4 \times \text{forward}(100) + \text{right}(90) = \text{perfect square.}$
■ Parameters	Changing the number inside forward() changes the shape size.
■ Debugging	Finding and fixing mistakes in code. Every programmer does it!

■ Homework — Practice at Home!

■ Draw a square on paper.

On each side write "forward(100)". At each corner write "right(90)". You just wrote a paper program!

■ Open Trinket.io at home.

Type the square code again from memory. Can you do it without looking at your notes?

■ Spot squares around you.

Look for square shapes at home — windows, tiles, books, TV screens. Count how many you find!

■ Try the hexagon challenge.

Use right(60) and repeat 6 times. Show us your hexagon next class!

■ Indian Cultural Inspiration

Your turtle art is proudly rooted in Indian artistic tradition! The geometric patterns you are creating with code are inspired by some of India's most beautiful art forms:



■ Rangoli	<i>Maharashtra, Gujarat & across India</i>	Geometric patterns made with coloured powder during festivals like Diwali. Artists move step by step, just like your turtle!
■ Kolam / Muggu	<i>Tamil Nadu, Andhra Pradesh, Kerala</i>	Mathematical floor art drawn at doorsteps every morning with rice flour. Perfect symmetry through careful, repeated movement.
■ Madhubani	<i>Bihar</i>	Folk paintings using geometric borders and frames — squares, triangles and diamonds arranged in beautiful repeating patterns.
■ Warli Art	<i>Maharashtra & Gujarat (tribal art)</i>	Ancient tribal art using circles, triangles and squares to tell stories. Pure geometry has always been at the heart of Indian art!

■ What's Next? — Session 2 Preview

■ <i>Add Colour!</i> <i>pencolor() & fillcolor()</i>	■ <i>Loops!</i> <i>Repeat shapes with for loops</i>	■ <i>Animations!</i> <i>Speed & moving patterns</i>
--	---	---

■ THANK YOU & KEEP CODING! ■

You are officially a Python programmer now. Every expert was once a beginner — and today you took your first step!